Final Project: Pick a Project

Computer Science - Week 10

Oct 3, 2026 - Version 0.0.2

Please make sure to your UD **email** AND **name** is below. Unless your UD emails are included in this table, then you will not earn any points for this assignment when it is graded! **Every person should submit their own individual worksheet**.

Talk about each of the questions with a friend or our LLM (or both), but **make sure the words you enter below are yours** (with the obvious exceptions notwithstanding). Brainstorming is best done in full sentences, and the main goal here is brainstorming. It's worth reviewing the information in this series of pages in the Drafter documentation to gain a better understanding of the process.

Role	Name	Email
Developer (You)	Ryan Miranda	rmiranda@udel.edu
Friend (optional)		

Part 1: The Goal & Prior Implementations

1. **Identify the Goal:** What specific goal are you trying to address? Think about your passions, hobbies, and academic interests. The latter are an excellent candidate to focus on. Some majors have suggestions listed below: these were provided by faculty in those departments. Other majors could consider a focus tied to a big final assignment in one of their recent courses in their major.
    - **Biomedical Engineering** majors may consider one of the following:
        - If you have started (or completed) the Cell/Tissue Design final project in BMEG100, design an organoid, which involves combining a cell type, biomaterial, and testing method. Have the website include a description and diagram of your design concept, a testing method, a needs statement, and an impact statement relevant to their organoid. Also include a link to a paper that supports your idea and a paragraph describing how that paper supports your design concept.

        - Otherwise, create a website related to your BMEG100 Lab Report 1, *Cytoskeletal and Nuclear Staining Lab Report Guidelines*, linked here (University of Delaware account required to access).
    - **Chemistry** and **Chemical Engineering** majors may consider creating an automated chemistry assistant that takes balanced reaction equations, starting reactant masses, and a harvested product amount to determine the limiting reactant, theoretical yield, and percent yield based on principles from Chemistry 2e (OpenStax) — Chapter 4: Chemical Reactions and Stoichiometry. To avoid manually compiling atomic mass tables or writing custom string tokenizers, use the ChemPy library to parse molecular formulas into exact molecular weights and verify reaction stoichiometry against compound data from the NIH PubChem Database. The program outputs an organized breakdown of limiting reagent status, product yields, and remaining masses of unreacted species. Also use the heat of the reaction to determine if it is exothermic or endothermic, and compute the heat flow per mole of the product.
    - Majors in the **Electric & Computer Engineering** department may consider a steganography project that hides messages in images, and then retrieves those messages from uploaded images.
    - **Mechanical Engineering** majors may consider a project that somehow uses both the NumPy and SciPy Python libraries.

I would be creating an application where you insert a type of tennis serve, and it returns where it landed, and whether or not the serve was in. Each serve is then logged to determine serve percentages.

2. **Prior Implementations:** Search the internet. What existing tools or websites currently attempt to address it?

The only websites that pop up are all simulators where you need to physically do it towards a projector, or stat trackers that track your in person performance.

3. **Your Implementation:** Briefly describe the legitimate purpose or functionality of your site. How would you want this web application to be used?

The purpose of this site is to determine whether an AI can perform equally as well as a real player when it comes to their serve going in. The AI is serving different serves with dependent and independent variables just like people would in real life.

Part 2: The Input-Process-Output (IPO) Model

Think about the fundamental flow of your program.

1. **The Core Goal:** Every program has three main phases: taking input, processing data, and presenting output. In one sentence, what is the ultimate goal your program will accomplish for the user? Be specific and use CS1 concepts.

The target audience will be tennis players who want to know if a specific type of serve will land in or out.

2. Input: What specific information will the user need to provide to your program? (e.g., a city name, a numerical budget, a list of hobbies).

The type of serve, side, left or right handed,

3. Processing: At a high level, how will the computer manipulate or evaluate that data? (e.g., calculate the average, filter out options that are too expensive, compare the input against a database).

A function will simulate the tennis serve using a loop. If statements will determine if it goes in or out. A dataclass will store each serve type. A list of serves will be kept in the Drafter State.

4. Output: What will the user get back? (e.g., a recommendation, a calculated safety factor, a formatted schedule).

The user will get where the ball landed in the service box, whether it was in or out, and the serve percentage for that specific type.

Part 3: Data Abstraction

Identify the "nouns" of your problem.

1. **Representing Reality:** Abstraction is the process of representing the real world inside a computer by ignoring unnecessary details. What are the core "things" (nouns) your program needs to track? (e.g., *If building a recipe app, the nouns are "Recipe" and "Ingredient"*).

Serve and Serve History are the two things my program is mainly tracking.

2. **Identifying Properties:** For the most important "thing" you listed above, what 3 to 4 specific details do you actually need to solve your problem, and what details will you abstract away?
    - *Details I need:* (e.g., *Ingredient name, weight, cost*)
    - *Details I will ignore:* (e.g., *Ingredient color, brand, expiration date*)

For Serve, 3 important things required are serve type, placement, and in or out. For Serve History, 4 important things required are the serve type, in/out, placement, and serves made from each type.

Part 4: Challenges, AI Strategy, & Goals

1. **Difficulties:** What parts of this project seem the most difficult or technically complex?

I would say the hardest part is getting the physics of the ball on the serve would be the harder part of the project. That along with tracking where each ball will land as it involves more independent variables.

2. **Coding Strategy:** If you will be using an LLM to help code, how will you ensure you understand the generated code and avoid "hallucinations" or "shallow learning" (copying without understanding)? If you won't be using an LLM to help code, this project might take about 4 hours a week to complete, so list each of the days and times in which you work on the project.

I will be using an LLM to help code. To avoid “shallow learning”, I will be documenting every piece that came from an LLM that way I am able to explain how it all works.

3. **Goals:** What is your ultimate goal for this project? (e.g., *Impress the graders with a creative UI, build something I can use in my other classes, learn Drafter deeply*).

The ultimate goal for this project is to show my student who I coach the serve percentages and the outcome for every type of serve. This way they can see what they believe is the best serve to hit on every first serve.

Part 5: Research & Brainstorming Conversation

Have a conversation with a Generative AI tool (like ChatGPT or Claude) OR a friend about your project idea. Do not ask for code. Instead, use the AI/friend as a sounding board to research your problem.

1. **The Initial Prompt/Question:** Ask the AI or your friend to help you brainstorm. (e.g., *"I want to build a simple web tool to help students track their study hours. What are 3 common problems students face when tracking their time, and what are 3 simple features a basic tool should have?"*). Paste your prompt below:

I want to build a simulation that tracks the type of tennis serve served and whether or not the ball goes in. From that data, it would calculate a serve percentage based on whether or not the ball landed in. What are 3 common problems that would change the serve and it not going in, and what would impact the serve percentage based on the ball's target?

2. **The Response:** Briefly summarize the best ideas the AI or your friend suggested. What existing solutions or prior apps did they mention?

3 common problems that cause misses are inconsistent toss, contact height and timing, too much pace for spin. Things that affect serve % based on the distance from the lines, serve type and target, depth target, first vs second serve, and natural forces such as wind, fatigue, pressure on the point.

3. **The Pushback:** You must reply to the AI/friend to narrow the scope when it's over-ambitious. Pick the most complex feature they suggested and explain why it is too complicated for an introductory programming project. Ask them for a simpler alternative. *Paste your response here:*

Both number 3 on common misses and distance from the line both are too complicated for an introductory programming project. Can you provide a simpler alternative to both of them?

4. **Reflection:** Based on this conversation, how has your project idea changed or become more focused?

This conversation has shown me what other factors I would need to include in my code. There are far more things that affect a serve from either going in or out that I did not account for. I will become more focused on each other factor that I did not take accountability beforehand.

Part 6: Resource Planning

1. What resources will you use if you get stuck?

Resources I will use if I get stuck are using the LLM, going to TA hours, or asking questions at the end of lectures.

2. If your LLM provides code that causes a Drafter or Bakery error, or if you run into a problem during deployment, what is your step-by-step debugging plan to resolve it?

I will identify what kind of error type it is and look at the line and surrounding lines to find the error. I will then try to fix it. If I can not I will use the LLM to try and have it help me correct it or identify where the issue is if I can not find it myself.

Part 7: LLM Conversation

If you used an LLM in any part of this work, copy and paste every part of that conversation below. Otherwise, you may be given a zero for the assignment. Obviously, these words will not all be yours, and this is the exception to the requirement at the top of this assignment that all responses be in your own words. If you did not use an LLM to assist you in this assignment, simply say so below.

Me: I want to build a simulation that tracks the type of tennis serve served and whether or not the ball goes in. From that data, it would calculate a serve percentage based on whether or not the ball landed in. What are 3 common problems that would change the serve and it not going in, and what would impact the serve percentage based on the balls target?

Claude: The useful way to frame this: every serve has an aim point plus some random scatter around it. Serve percentage is the share of that scatter that lands inside the service box. Problems widen or shift the scatter. Target choice decides how much of the scatter fits

inside the lines.

3 common problems that cause misses

1. **Inconsistent toss.** The toss sets everything else. If it drifts too far forward, the player swings down and the ball tends to hit the net. If it's too far back or behind the head, the ball usually sails long. If it drifts sideways, the ball goes wide, or the player compensates and loses control. In a sim, toss error adds a directional bias to both depth and width.
2. **Contact height and timing.** Hitting the ball after it has dropped, or swinging early or late, changes the launch angle. A lower contact point leaves a smaller window for clearing the net and still landing in the box. This matters most on flat serves. Model it as error in launch angle, which mostly affects depth (net vs. long).
3. **Too much pace for the spin.** Spin is what pulls the ball down into the box. A flat serve hit hard has very little margin. A kick or topspin serve can clear the net by several feet and still dip in. If a player hits harder without adding spin, especially when nervous or tired, the ball goes long. Model this as speed vs. spin: more speed widens the depth scatter, and more spin narrows it.

What affects serve % based on the target

- **Distance from the lines:** Aiming at the T or out wide puts the aim point close to a line, so part of the scatter is guaranteed to land out. Aiming at the body or the middle of the box gives margin on every side and produces the highest percentage.
- **Serve type × target:** Flat serves have the tightest margin, so they're riskiest to the corners. Slice curves sideways, which works well out wide on the deuce side but can drift out if over-hit. Kick serves are the safest overall, which is why most second serves are kicks.
- **Depth target:** Aiming deep near the service line wins more points but loses percentage. Aiming shorter gains margin, but the serve is easier to attack.
- **First vs. second serve:** Players usually take more risk on first serves (flatter, closer to the lines) and play safer on second serves. Expect roughly 60–65% on firsts and 85–90%+ on seconds.
- **Outside factors (optional):** Wind, fatigue late in a match, and pressure points all widen the scatter.

How to turn it into a sim

1. Court: The service box is 21 ft deep × 13.5 ft wide. The net is 3 ft high at the center.
2. Each serve record: serve type (flat, slice, kick), target (T, body, wide), side (deuce, ad), first or second serve, and result (in, net, long, wide).
3. Landing spot: landing = target + bias from problems + random noise, using a normal distribution in x and y. Each serve type gets its own spread.
4. Serve %: in / total. You can break it down by type, target, and first vs. second serve, which is where the interesting patterns will show up.

Me: rewrite the affects based on the serve going in

Claude: *What affects whether the serve goes in, based on the target*

- **Distance from the lines:** The farther the target is from the lines, the more room the ball has to go in. Aiming at the body or the middle of the box keeps almost the whole scatter inside the box, so these serves go in most often. Aiming at the T or out wide puts the target close to a line, so fewer serves land in.
- **Serve type × target:** Kick serves go in most often because topspin pulls the ball down into the box, even with high net clearance. Slice serves go in reliably out wide on the deuce side because the curve carries the ball into the box. Flat serves go in least often, especially to the corners, because they need a near-perfect launch angle to clear the net and still land short of the service line.
- **Depth target:** A target a few feet inside the service line gives the ball room to clear the net and still drop in. A target right at the service line means any small error in launch angle sends the ball long.
- **First vs. second serve:** Second serves go in more often (about 85–90%+) because players choose safer targets and add spin. First serves go in less often (about 60–65%) because players aim closer to the lines with more pace.
- **Outside factors (optional):** Calm conditions and a fresh, relaxed server keep the scatter tight, so more serves land in. Wind, fatigue, and pressure points widen the scatter, so fewer serves go in.

Me: Both number 3 on common misses and distance from the line both are too complicated for an introductory programming project. Can you provide a simpler alternative to both of them?

Claude: *Simpler #3: Rushing the serve.* When a player rushes, especially after missing a first serve, they skip their routine and the serve goes in less often. In the sim, this is a yes/no flag:

- `rushed = True` subtracts a fixed amount from the chance of the serve going in (for example, 15%).
- `rushed = False` leaves the chance unchanged.

This needs no physics or speed and spin math. It's one `if` statement.

Simpler "distance from the lines": a base chance for each target. Instead of modeling where the ball lands, give each target a fixed chance of going in. Safer targets get higher numbers:

Target	Chance to go in

Body	80%
T	65%
Wide	60%

In Python, that's a dictionary like `{"body": 0.80, "T": 0.65, "wide": 0.60}`.

How they fit together. For each serve:

1. Look up the target's base chance.
2. If the serve was rushed (or had a toss or contact problem), subtract a set amount.
3. Generate `random.random()`. If it's less than the chance, the serve is in; otherwise it's out.
4. Count the ins and divide by the total to get the serve percentage.

The whole simulation becomes a loop, a dictionary, a few `if` statements, and a counter.

Part 8: Submit

In Google docs, with this document open, please click on *File* > *Download* > *Microsoft Word (.docx)*, and make sure you put that download somewhere you can find it again. Then, upload it to the Canvas assignment about selecting or brainstorming your final project.

When you're done with this assignment, review this page of the Drafter documentation to better prepare for the next step.